

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Francisca Zavala Cortés

2 de abril de 2026

10:13

1. Entorno experimental

Se detalla a continuación la infraestructura utilizada en el proyecto:

El hardware consiste en Procesador Intel(R) Core(TM) Ultra 5 125H (3.60 GHz); 16 GB de memoria RAM instalada; Almacenamiento en SSD NVMe.

El entorno de software corresponde a Window 11 Home Single Language 24H2, ejecutando Ubuntu 24.04.2 (WSL 2), compilador g++ 13.3.0 con estándar C++17. Los scripts de generación de casos de prueba, y generación de gráficos fueron escritos en Python 3.12.3, y se ejecutan mediante un Makefile.

Para el experimento, solo se considera los casos de prueba dados en el enunciado, los cuales son generados por los archivos `arraygenerator.py`, y `matrixgenerator.py`.

- **Ordenamiento:** Los datos son números de tipo entero, el tamaño de la muestra es: $n = 10^1, 10^3, 10^5, 10^7$. Los arreglos son de tipo: ascendente, descendente, y aleatorio.
- **Multiplicación de matrices:** Los datos son números de tipo entero, la muestra incluye matrices $n \times n$, donde: $n = 2^4, 2^6, 2^8, 2^{10}$. Las matrices de tipo: densas, diagonales, y dispersas.

2. Resultados y Análisis

El tiempo de ejecución y uso de memoria medidos son registrados en archivos de texto. Con ayuda de los scripts `plotgenerator.py`, se generan los gráficos de los resultados, ejecutando “make plot”, definido en el Makefile.

Resultados Ordenamiento de Arreglos:

Utilizando gráficos, se muestra los resultados principales obtenidos, comparando los tres algoritmos para distintos tipos de entradas. En el eje X se representa el Tamaño de entrada n , y en el eje Y se representa Tiempo de ejecución (ms), o uso de memoria (bytes), dependiendo del tipo de gráfico.

Se analizan todos los casos de prueba con datos ordenados de forma ascendente, aleatoria y descendente, con los que se obtiene los siguientes tiempos de ejecución promedio:

Cabe notar que para 10.000.000 no hay datos medidos de tiempo de ejecución, ya que después de las 7

Algoritmo	MergeSort(ms)	QuickSort(ms)	Sort(ms)
10	0.003	0.001	0.0009
1000	0.33	0.1	0.02
100000	1505.27	568.85	1.33
10 000 000	-	-	-
-	-	-	-

Cuadro 1: Tiempo de ejecución promedio para ordenamiento de arreglos

horas se detiene la ejecución, y se considera como caso outlier.

Es posible notar con el cuadro 1, que el algoritmo que en promedio se demora el mayor tiempo es MergeSort, y el algoritmo que es más eficiente es el Sort, de la librería C++.

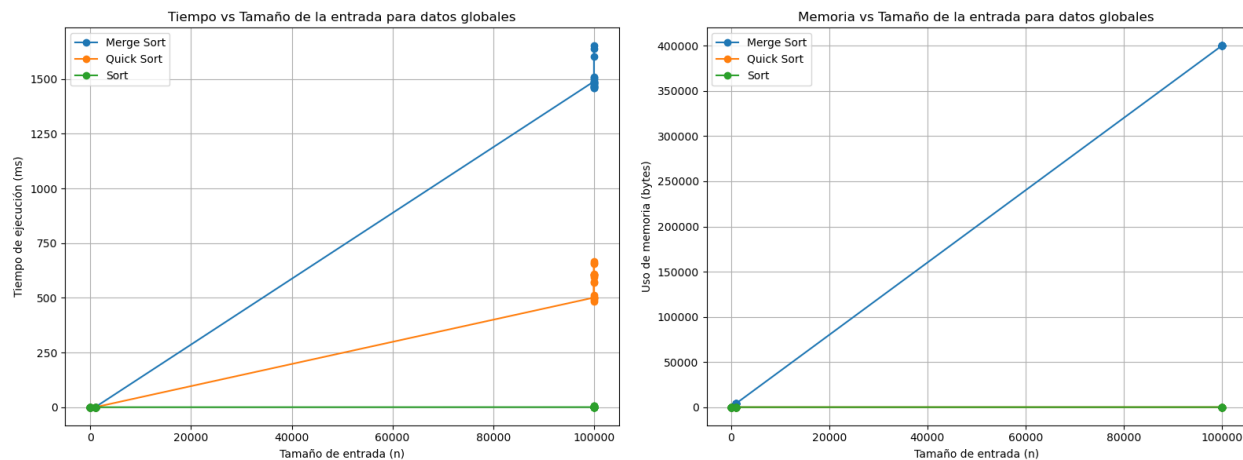


Figura 1: Gráfico de tiempo de ejecución y uso de memoria para ordenamiento de arreglos.

Con los gráficos se puede notar que la memoria utilizada es igual para los algoritmos QuickSort y Sort. Además se corrobora lo mencionado sobre el cuadro 1, siendo MergeSort la curva con mayor tiempo, y Sort, la curva con tiempo más cercano a 0.

Resultados Multiplicación de Matrices:

Utilizando gráficos, se muestra los resultados principales obtenidos, comparando los dos algoritmos para distintos tipos de entradas. En el eje X se representa el Tamaño de la matriz ($n \times n$), y en el eje Y se representa Tiempo de ejecución (ms), o uso de memoria (bytes), dependiendo del tipo de gráfico.

Considerando todos los casos de prueba con matrices densas, dispersas y diagonales, se obtiene los siguientes tiempos de ejecución promedio:

Se puede ver en la Tabla 2, que, el algoritmo que demora más tiempo es Strassen, y el algoritmo que

Tamaño ($n \times n$)	Naive(ms)	Strassen(ms)
16	0.005	2.07
64	0.26	114.23
256	16.27	4493.9
1024	1097.67	141855.75

Cuadro 2: Tiempos de ejecución promedio para Multiplicación de Matrices

demora menos es Naive.

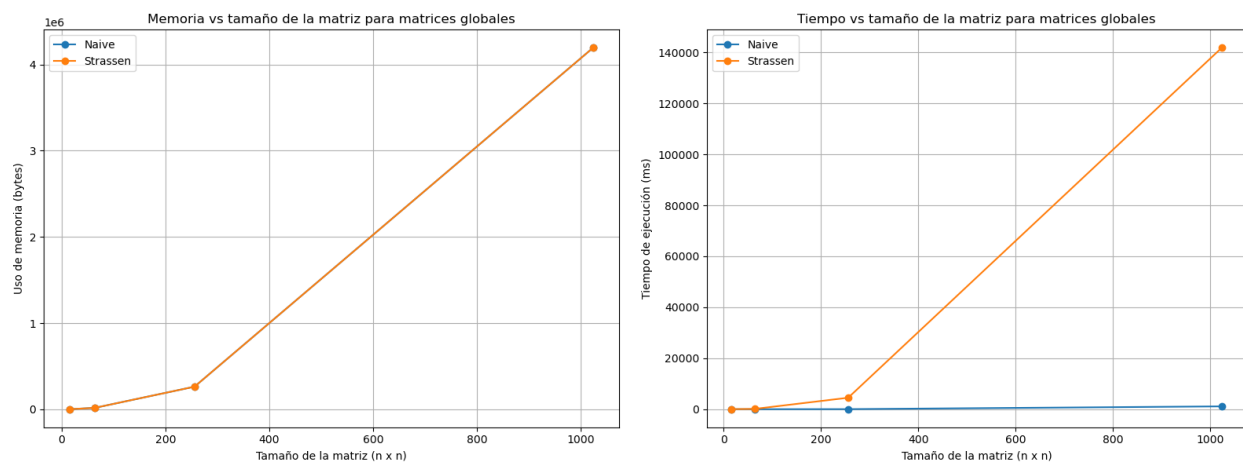


Figura 2: Gráfico de tiempo de ejecución y uso de memoria para Multiplicación de Matrices.

Con los gráficos se puede notar que la memoria utilizada es igual para ambos algoritmos, y se corrobora lo mencionado sobre el cuadro 2, siendo Strassen la curva con mayor tiempo, y Naive, la curva con tiempo menor.

3. Conclusiones

En este informe se pudo determinar diferencias entre la complejidad asintótica teórica y su comportamiento en la práctica, mediante la obtención de resultados replicables, y un análisis detallado.

Para los algoritmos de ordenamiento de arreglos, es posible confirmar la hipótesis de que el algoritmo más eficiente, es decir, el que se demora el menor tiempo para ejecutarse, es el algoritmo Sort, para todos los tipos de entrada. Y, para el algoritmo menos eficiente, se tiene el algoritmo MergeSort.

Para los algoritmos de multiplicación de matrices, el algoritmo Naive es consistentemente más eficiente para todos los tipos de matrices de entrada. Lo que significa que el algoritmo Strassen tiene el peor rendimiento de los dos.

En conclusión, con los resultados se evidencian diferencias significativas entre la complejidad asintótica teórica y su desempeño en la práctica. La sobrecarga asociada a la gestión de memoria y la recursividad puede contrarrestar las ventajas teóricas en escenarios reales, y solo un análisis experimental riguroso, con resultados replicables y un examen detallado, permite identificar la opción más eficiente en cada contexto.